

# Motion Control #3: QP-Based Reactive Control

## Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 16, 2026

# 1. Fundamentals: What is Quadratic Programming (QP)?

Quadratic Programming is a special type of mathematical optimization problem used to minimize a quadratic objective function subject to linear constraints.

## General Form of a QP:

$$\text{minimize } f(x) = \frac{1}{2}x^T Hx + g^T x$$

subject to  $Ax = b$  (Equality Constraints)

and  $Cx \leq d$  (Inequality Constraints)

- **H (Hessian)**: Must be positive semi-definite (PSD) for the problem to be convex.
- **Efficiency**: Convex QPs can be solved extremely fast (often  $< 1\text{ms}$ ) using specialized solvers (e.g., OSQP, qpOASES), making them ideal for real-time robot control.

## 2. Relationship: QP and the Pseudo-inverse

Consider the unconstrained **Least Squares Program (LSP)**:

$$\min_x \frac{1}{2} \|Ax - b\|^2 = \min_x \frac{1}{2} (Ax - b)^T (Ax - b)$$

Expanding this, we get:

$$F(x) = \frac{1}{2} x^T (A^T A) x - (A^T b)^T x + \frac{1}{2} b^T b$$

This is a QP where  $H = A^T A$  and  $g = -A^T b$ . Setting the gradient  $\nabla F = 0$ :

$$A^T A x - A^T b = 0 \implies x^* = (A^T A)^{-1} A^T b$$

**Insight:** The **Left Pseudo-inverse** ( $A^\dagger$ ) is simply the analytical solution to a basic, unconstrained QP.

### 3. Weighted Optimization and Metrics

In robotics, not all errors or joint movements are equal. We use a **Weight Matrix ( $W$ )** to prioritize certain dimensions or reflect physical properties.

**Weighted Least Squares QP:**

$$\min_x \frac{1}{2} \|Ax - b\|_W^2 = \min_x \frac{1}{2} (Ax - b)^T W (Ax - b)$$

The analytical solution becomes the **Weighted Pseudo-inverse ( $A_W^\dagger$ )**:

$$x^* = (A^T W A)^{-1} A^T W b \quad \text{or} \quad x^* = W^{-1} A^T (A W^{-1} A^T)^{-1} b$$

- If  $W = M^{-1}$  (Inverse of Mass Matrix), this leads to **Dynamically Consistent** solutions.
- This is the mathematical bridge between pure kinematics and Operational Space Control.

## 4. Joint-Space Inverse Dynamics as a QP

Traditional Inverse Dynamics:  $\tau = M\ddot{q}^d + h$ . In the QP framework, we treat the Dynamics as a constraint:

**The Problem:**

$$\min_{\ddot{q}, \tau} \frac{1}{2} \|\ddot{q} - \ddot{q}^d\|^2$$

$$\text{s.t. } M\ddot{q} + h = \tau$$

$$\text{and } -\tau^{\max} \leq \tau \leq \tau^{\max} \quad (\text{Torque Limits})$$

- Without limits, the solution is the standard ID.
- With limits, the QP solver automatically finds the "closest" feasible acceleration.

## 5. Generalizing Tasks and Underactuation

We can define any task (position, velocity, or force) as an affine function  $Az - b = 0$ .

**Underactuated Systems:** For robots like legged systems, not all joints are actuated. We use a **\*\*Selection Matrix ( $S$ )\*\***:

$$M\ddot{q} + h = S^T \tau + J^T F$$

where  $S = [0 \quad I]$  filters out the unactuated degrees of freedom (e.g., the floating base).

**Rigid Contacts:** Contact points impose kinematic constraints:  $J\ddot{q} + \dot{J}\dot{q} = 0$ . The QP now solves for  $z = (\ddot{q}, \tau, F)$  simultaneously.

## 6. Task-Space Inverse Dynamics (TSID)

TSID is a unified framework that solves for joint accelerations and torques simultaneously to satisfy task-space objectives.

**Problem Formulation:** Instead of solving for  $\ddot{q}$  and  $\tau$  sequentially, we define a combined decision variable  $z$ :

$$z = \begin{bmatrix} \ddot{q} \\ \tau \end{bmatrix} \in \mathbb{R}^{2n}$$

**Optimization Objective (End-effector Control):**

$$\text{minimize } \frac{1}{2} \|Az - b\|^2$$

where  $A = \begin{bmatrix} J & 0 \end{bmatrix}$  and  $b = \ddot{x}^d - J\dot{q}$ .

**Consistency Constraint (Robot Dynamics):** The solution must satisfy the rigid body dynamics at all times:

$$\begin{bmatrix} M(q) & -I \end{bmatrix} z = -h(q, \dot{q})$$

- This formulation allows us to minimize task errors while strictly enforcing the physical consistency of the robot.

## 7. Full Task-Space Inverse Dynamics (TSID)

For complex robots in contact, TSID extends its decision variables to include contact forces.

**Decision Variables:**  $z = (\ddot{q}, \tau, F) \in \mathbb{R}^{n+n+3k}$

- $\ddot{q}$ : Joint accelerations.
- $\tau$ : Joint torques.
- $F$ : Contact forces (decision variables of the QP).

**Constraints:**

- **Dynamics:**  $M\ddot{q} + h = \tau + J^T F$ .
- **Underactuation:**  $S^T \tau$  where  $S = [0 \quad I]$  for floating base.
- **Contact Consistency:**  $J\ddot{q} + \dot{J}\dot{q} = 0$  (no-slip condition).
- **Friction Cones:** Linearized pyramidal cone  $Bf \leq 0$ .

## 7. Full Task-Space Inverse Dynamics (TSID)

When we have multiple tasks  $g_i(z) = \|A_i z - b_i\|^2$ , we can sum them using weights  $w_i$ .

$$\min_z \sum_{i=1}^N w_i \|A_i z - b_i\|^2$$

subject to Physical Constraints

- **Pro:** Single QP, very fast to solve.
- **Con:** Weight tuning is "voodoo" (heuristic). High-priority tasks can be "pushed" by lower-priority tasks if weights are not perfectly balanced.

## 8. Hierarchical Quadratic Programming (HQP)

HQP (or Lexicographic QP) solves tasks in a **\*\*strict hierarchy\*\*** (Hard Priorities).

**Algorithm Flow:** For  $i = 1$  to  $N$ :

$$\begin{aligned} & \min_z ||A_i z - b_i||^2 \\ & \text{s.t. } A_k z = A_k z_k^* \quad \forall k < i \quad (\text{Protect Higher Tasks}) \\ & \quad \text{and Physical Constraints} \end{aligned}$$

**Comparison:**

- **Weighted QP:** A "democracy" where everyone has a vote, but some have more money (weights).
- **HQP:** A "monarchy" where the King (Task 1) is never disturbed by the Peasant (Task N).

## 9. Comparison: TSID, Weighted QP, and HQP

Choosing the right strategy depends on the robot's hardware and the task's criticality.

| Feature       | Basic TSID             | Weighted TSID             | HQP                  |
|---------------|------------------------|---------------------------|----------------------|
| Priority      | Single Task            | <b>Soft</b> (Weights)     | <b>Hard</b> (Strict) |
| Constraint    | Rigid Dynamics         | Dynamics + Limits         | Dynamics + Limits    |
| Decision Var. | $z = (\ddot{q}, \tau)$ | $z = (\ddot{q}, \tau, F)$ | $z_i$ (Sequential)   |
| Conflict      | N/A                    | Tasks "Compromise"        | Higher Task Wins     |
| Use Case      | Basic Manipulators     | Reactive Behaviors        | Humanoids / Legged   |

### Summary Recommendation:

- Use **Weighted QP** for fast, reactive behaviors where tasks can "compromise" .
- Use **HQP** when safety tasks (e.g., self-collision) must **never** be disturbed by tracking tasks.